

SIMULASI REALISTIS

Git & GitHub Team Workflow

Dari onboarding developer sampai Pull Request, code review, CI, conflict, approval, squash merge, hotfix, dan sinkronisasi tim.

Mode latihan: 1 akun untuk pemanasan, lalu 2 akun untuk simulasi reviewer/developer yang benar-benar terpisah.

ROLE A - Tech Lead / Maintainer

ROLE B - Developer

Repository: acme-profile-app

Task utama: APP-101 Add editable user profile

Hotfix: APP-102 Fix empty display name

Target akhir:

main <- Pull Request <- feature branch <- commits lokal

Catatan: nama perusahaan, akun, ticket, dan source code di modul ini adalah simulasi. Workflow dibuat menyerupai pola kerja tim software modern, tetapi aturan persis setiap perusahaan dapat berbeda.

1. Berapa akun yang dibutuhkan?

Minimal: 1 akun GitHub. Ini cukup untuk memahami branch, commit, push, Pull Request, dan merge. Namun satu akun tidak memberi pengalaman review antarpengguna yang benar-benar terpisah.

Untuk latihan realistis: 2 akun. Akun A menjadi Tech Lead/Maintainer, akun B menjadi Developer. Gunakan akun latihan milikmu sendiri dan ikuti aturan GitHub; jangan memakai identitas orang lain.

Mode	Akun	Yang bisa dilatih
Pemanasan	1	Branch, commit, push, PR, merge
Realistis	2	Permission, reviewer, request changes, approval, protected main
Tim asli	3+	Parallel work, ownership, review silang, konflik lebih alami

Pembagian role latihan

Role	Tanggung jawab
Tech Lead / Maintainer	Membuat repo, mengatur aturan main, membuat issue/task, review PR, approve/merge.
Developer	Clone, membuat feature branch, coding, test, commit, push, membuka PR, memperbaiki feedback

Prinsip utama: developer tidak bekerja langsung di main. Perubahan masuk melalui branch dan Pull Request.

2. Skenario project yang akan kita praktikkan

Kamu bergabung ke tim kecil bernama Acme App Team. Produk mempunyai halaman profile sederhana. Tech Lead membuat ticket APP-101: pengguna harus bisa menyimpan display name dan bio.

```
acme-profile-app/  
├── README.md  
├── src/  
│   └── profile.js  
└── test/  
    └── profile.test.js
```

Definition of Done APP-101: fungsi menerima display name dan bio; display name wajib diisi; whitespace dibersihkan; test lulus; tidak ada direct push ke main; PR mendapat review sebelum merge.

3. Fase 0 - Persiapan komputer Developer

Pastikan Git tersedia. Jalankan command berikut dari terminal.

```
git --version
git config --global user.name "Nama Kamu"
git config --global user.email "email-yang-kamu-pakai-untuk-git"
```

```
git config --global --get user.name
git config --global --get user.email
```

Gunakan email milikmu sendiri. Untuk autentikasi GitHub, ikuti metode yang tersedia pada akunmu (misalnya HTTPS credential flow atau SSH). Jangan menaruh token/password di source code atau commit.

Kebiasaan yang wajib sejak awal

```
git status
git diff
git log --oneline --graph --decorate --all
```

Tiga command itu akan sering dipakai. status menunjukkan kondisi kerja, diff menunjukkan perubahan, dan log menunjukkan sejarah branch.

4. Fase 1 - Tech Lead membuat repository

Login sebagai Akun A. Buat repository bernama acme-profile-app. Untuk latihan pertama, repository boleh private atau public sesuai kebutuhanmu. Tambahkan Developer (Akun B) sebagai collaborator bila diperlukan.

Kemudian Tech Lead menyiapkan project awal. Contoh isi src/profile.js:

```
function createProfile(displayName, bio) {
  return { displayName, bio };
}

module.exports = { createProfile };
```

Contoh test awal test/profile.test.js dapat disesuaikan dengan framework yang kamu pakai. Fokus modul ini adalah workflow Git; project JavaScript hanya kendaraan latihan.

Commit awal Tech Lead

```
git add .
git commit -m "chore: initialize profile app"
git push origin main
```

5. Fase 2 - Aturan main seperti project tim

Pada repository tim, branch utama biasanya diberi rules/protection. Nama menu GitHub dapat berubah dari waktu ke waktu, jadi fokus pada tujuan aturannya.

Aturan	Tujuan
Require Pull Request	Perubahan ke main harus melalui PR.
Require approval	Minimal reviewer menyetujui perubahan.
Require status checks	Test/lint yang diwajibkan harus sukses.
Require conversation resolution	Komentar review yang diwajibkan harus diselesaikan.
Block force push	Mengurangi risiko history main ditulis ulang secara tidak sengaja.

Untuk latihan dua akun, atur agar Akun B tidak bisa melewati proses review. Jika fitur tertentu tidak tersedia pada jenis repository/plan yang kamu gunakan, tetap lakukan prosesnya secara manual: jangan merge sebelum review dan checks selesai.

6. Fase 3 - Tech Lead membuat ticket APP-101

Buat Issue atau catatan task dengan spesifikasi realistis berikut.

APP-101 - Add editable user profile

Acceptance Criteria

- User profile has displayName and bio
- displayName cannot be empty
- Leading/trailing spaces are removed
- Existing behavior is not broken
- Tests pass

Out of Scope

- Avatar upload
- Database integration
- Authentication

Ini penting: developer harus mengetahui apa yang termasuk dan tidak termasuk scope. PR yang terlalu besar lebih sulit direview.

7. Fase 4 - Developer clone dan membuat branch

Sekarang pindah ke Akun B sebagai Developer. Clone repository yang dibuat Tech Lead.

```
git clone <URL-REPOSITORY>
cd acme-profile-app
```

```
git remote -v
git status
git branch --show-current
```

Sebelum mulai task, sinkronkan main lalu buat branch dari main terbaru.

```
git switch main
git pull origin main
git switch -c feature/APP-101-editable-profile
```

Periksa:

```
git branch
# * feature/APP-101-editable-profile
# main
```

Kenapa branch dibuat setelah pull? Supaya titik awal feature berasal dari main terbaru, bukan snapshot lama di laptop.

8. Fase 5 - Coding commit pertama

Developer mengimplementasikan normalisasi input. Ubah `src/profile.js`:

```
function createProfile(displayName, bio = "") {
  const cleanName = displayName.trim();
  const cleanBio = bio.trim();

  return {
    displayName: cleanName,
    bio: cleanBio,
  };
}
```

```
module.exports = { createProfile };
```

Sebelum commit, jangan langsung `git add` . secara buta. Inspeksi perubahan.

```
git status
git diff
```

Kalau sesuai scope:

```
git add src/profile.js
git diff --staged
git commit -m "feat: normalize profile fields"
```

9. Fase 6 - Commit kedua: validation + test

Acceptance criteria menyatakan display name tidak boleh kosong. Tambahkan validation.

```
function createProfile(displayName, bio = "") {
  const cleanName = displayName.trim();
  const cleanBio = bio.trim();

  if (!cleanName) {
    throw new Error("displayName is required");
  }

  return { displayName: cleanName, bio: cleanBio };
}
```

```
module.exports = { createProfile };
```

Tambahkan test sesuai test runner project. Secara konseptual, test harus mencakup nama normal, trimming, dan empty name. Setelah test lokal berhasil:

```
git status
git diff
# jalankan test project-mu

git add src/profile.js test/profile.test.js
git diff --staged
git commit -m "feat: validate required display name"
```

History sekarang

```
git log --oneline --decorate -5

<commit-2> feat: validate required display name
<commit-1> feat: normalize profile fields
<initial>  chore: initialize profile app
```

10. Fase 7 - Push feature branch

```
git push -u origin feature/APP-101-editable-profile
```

Flag -u menghubungkan local branch dengan remote tracking branch. Push berikutnya biasanya cukup `git push`.

Pada titik ini, main belum berubah. Hanya remote feature branch yang mendapat commit developer.

11. Fase 8 - Developer membuka Pull Request

Di GitHub, buat Pull Request dengan base main dan compare/head feature/APP-101-editable-profile.

Contoh PR realistis

Title

APP-101: add editable user profile validation

Summary

- Normalize displayName and bio
- Reject empty displayName
- Add tests for profile validation

How to Test

1. Run project tests
2. Create profile with " Raka "
3. Confirm stored displayName is "Raka"
4. Create profile with whitespace-only name
5. Confirm validation error occurs

Out of Scope

- Avatar upload
- Persistence/database

Related

Closes APP-101

Sebelum meminta review, Developer melakukan self-review pada tab perubahan: cari debug code, file tidak sengaja ikut, secret, typo, atau perubahan di luar scope.

12. Fase 9 - CI/check otomatis

Project tim sering menjalankan checks ketika PR dibuat. Contoh pipeline sederhana: checkout source, install dependency, lint, lalu test.

Pull Request #14

Checks

[PASS] lint
[PASS] unit-tests
[PASS] build

Review

[WAIT] maintainer approval

Jika test gagal, jangan meminta reviewer mengabaikannya. Buka log, pahami kegagalannya, perbaiki di branch yang sama, commit, lalu push. PR akan diperbarui.

13. Fase 10 - Review pertama oleh Tech Lead

Akun A membuka PR sebagai reviewer. Reviewer tidak hanya melihat apakah code "jalan", tetapi apakah perubahan memenuhi requirement dan mudah dipelihara.

Contoh komentar review

src/profile.js

Reviewer:
"displayName.trim() akan error bila displayName bukan string.
Bisakah input invalid ditangani secara eksplisit dan dites?"

Review result: REQUEST CHANGES

Ini bukan kegagalan. Request changes adalah bagian normal dari kolaborasi. Developer tidak membuat PR baru; ia memperbaiki branch PR yang sama.

14. Fase 11 - Developer menangani review

Kembali sebagai Akun B. Pastikan berada di feature branch.

```
git switch feature/APP-101-editable-profile
git status
```

Perbaiki validation:

```
function createProfile(displayName, bio = "") {
  if (typeof displayName !== "string") {
    throw new Error("displayName must be a string");
  }

  const cleanName = displayName.trim();
  const cleanBio = typeof bio === "string" ? bio.trim() : "";

  if (!cleanName) {
    throw new Error("displayName is required");
  }

  return { displayName: cleanName, bio: cleanBio };
}
```

Tambahkan test input invalid, jalankan test, lalu:

```
git add src/profile.js test/profile.test.js
git diff --staged
git commit -m "fix: handle invalid profile input"
git push
```

PR yang sama sekarang mempunyai commit baru. Balas komentar review secara singkat, misalnya: Handled invalid displayName input and added coverage. Lalu reviewer memeriksa ulang.

15. Fase 12 - Main berubah saat PR masih direview

Sekarang kita buat kondisi realistis: Developer lain/Tech Lead sudah merge perubahan lain ke main. Feature branch APP-101 menjadi tertinggal.

Sebelum:

```
A---B main
    \---C---D feature/APP-101
```

Setelah perubahan lain masuk:

```
A---B---E---F main
    \---C---D feature/APP-101
```

Developer mengambil informasi remote:

```
git fetch origin
git log --oneline --graph --decorate --all
```

Pilihan A - merge main ke feature

```
git switch feature/APP-101-editable-profile
git merge origin/main
```

Pilihan ini tidak menulis ulang commit feature, tetapi dapat membuat merge commit.

Pilihan B - rebase feature ke main

```
git switch feature/APP-101-editable-profile
git rebase origin/main
```

Rebase menghasilkan history linear tetapi menulis ulang commit feature. Pada branch yang sudah dipush, ikuti kebijakan tim sebelum force-push. Jangan melakukan rebase/force-push pada branch bersama tanpa memahami dampaknya.

16. Fase 13 - Simulasi merge conflict yang nyata

Tech Lead mengubah baris return di `profile.js` pada main, sementara feature branch mengubah area yang sama. Saat integrasi, Git tidak bisa memilih otomatis.

```
<<<<<< HEAD
return { displayName: cleanName, bio: cleanBio };
=====
return { displayName: cleanName, bio: cleanBio, updatedAt: Date.now() };
>>>>>> origin/main
```

Jangan sekadar menghapus marker. Pahami maksud kedua perubahan. Misalnya requirement main memang membutuhkan `updatedAt`, maka hasil akhir dapat mempertahankan validation feature sekaligus field tersebut.

```
return {
  displayName: cleanName,
  bio: cleanBio,
  updatedAt: Date.now(),
};
```

Jika kamu sedang merge

```
git add src/profile.js
git commit
git push
```

Jika kamu sedang rebase

```
git add src/profile.js
git rebase --continue
# setelah rebase branch yang sudah pernah dipush, ikuti kebijakan tim
# dan gunakan force-with-lease hanya bila memang diperlukan/diizinkan
```

Rule keselamatan: jangan memakai `git push --force` sembarangan. Pada workflow yang memerlukan force setelah rebase, `--force-with-lease` lebih aman karena memeriksa apakah remote berubah tanpa sepengetahuanmu.

17. Fase 14 - Approval dan merge

Setelah conflict selesai dan checks kembali hijau, Tech Lead melakukan review ulang.

Checks

```
[PASS] lint
[PASS] unit-tests
[PASS] build
```

Review

```
[PASS] Maintainer approved
```

Conversations

```
[PASS] resolved
```

Status

```
READY TO MERGE
```

Untuk latihan ini gunakan Squash and merge. Semua commit kecil pada feature digabung menjadi satu commit representatif di main.

feature branch commits:

- feat: normalize profile fields
- feat: validate required display name
- fix: handle invalid profile input
- merge/rebase resolution as applicable

main after squash:

- APP-101: add editable user profile validation (#14)

Squash cocok untuk latihan ini karena main menjadi ringkas: satu perubahan produk/PR direpresentasikan oleh satu commit. Tim lain dapat memilih merge commit atau rebase merge sesuai kebijakan.

18. Fase 15 - Setelah PR di-merge

Developer jangan terus bekerja di branch lama. Sinkronkan main lokal.

```
git switch main
git pull origin main
git log --oneline --graph --decorate -10
```

Jika feature branch sudah tidak diperlukan:

```
git branch -d feature/APP-101-editable-profile
```

remote branch dapat dihapus melalui GitHub atau:

```
git push origin --delete feature/APP-101-editable-profile
```

Sekarang satu siklus task selesai: ticket -> branch -> code -> commits -> push -> PR -> CI -> review -> revision -> approval -> merge -> sync.

19. Simulasi kedua - Hotfix APP-102

Setelah APP-101 masuk main, QA menemukan bug: display name yang sangat panjang membuat UI bermasalah. Tech Lead membuat APP-102: batasi display name maksimal 50 karakter.

Developer memulai dari main terbaru

```
git switch main
git pull origin main
git switch -c fix/APP-102-display-name-length
```

Implementasikan validation dan test. Lalu:

```
git status
git diff
# jalankan tests
```

```
git add src/profile.js test/profile.test.js
git commit -m "fix: limit profile display name length"
git push -u origin fix/APP-102-display-name-length
```

Buka PR baru karena APP-102 adalah task terpisah. Jangan memasukkan refactor besar atau fitur avatar ke hotfix ini.

Review checklist hotfix

Cek	Pertanyaan
Scope	Apakah hanya memperbaiki APP-102?
Regression	Apakah behavior APP-101 masih bekerja?
Boundary	Apakah 50 karakter diterima dan 51 ditolak?
CI	Apakah test dan lint lulus?
Risk	Apakah perubahan kecil dan mudah direvert?

20. Praktik 2 akun - urutan klik dan terminal

Gunakan tabel ini sebagai naskah role-play. Selesaikan satu baris sebelum pindah ke baris berikutnya.

#	Akun/Role	Aksi
1	A - Lead	Buat repo + initial commit.
2	A - Lead	Tambahkan B sebagai collaborator bila diperlukan.
3	A - Lead	Siapkan aturan agar main melalui PR/review jika tersedia.
4	A - Lead	Buat task APP-101.
5	B - Dev	Clone repo, pull main, buat feature branch.
6	B - Dev	Coding, test, 2 commit terarah.
7	B - Dev	Push branch dan buka PR.
8	A - Lead	Review dan Request Changes.
9	B - Dev	Perbaiki pada branch yang sama, commit, push.
10	A - Lead	Buat perubahan lain di main untuk simulasi branch tertinggal.
11	B - Dev	Fetch + integrasikan main; selesaikan conflict bila ada.
12	A - Lead	Review ulang dan Approve.
13	A - Lead	Squash and merge.
14	B - Dev	Pull main terbaru dan hapus feature branch.

21. Apa yang harus kamu screenshot sebagai bukti latihan?

Kalau latihan ini dipakai sebagai portofolio belajar pribadi, simpan bukti yang tidak mengandung secret atau data sensitif: graph branch sebelum merge, halaman PR, satu contoh review comment, checks yang lulus, dan history main setelah squash merge.

Jangan screenshot token, credential, email pribadi yang tidak ingin dibagikan, repository secret, atau halaman pengaturan sensitif.

22. Kesalahan yang sengaja harus kamu hindari

Kesalahan	Perbaiki
Coding langsung di main	Buat branch per task.
git add . tanpa melihat diff	Cek status/diff; stage hanya file relevan.
Commit "update" / "fix"	Gunakan pesan yang menjelaskan intent.
PR raksasa	Pisahkan berdasarkan scope/task.
Bikin PR baru saat request changes	Perbaiki branch PR yang sama.
Resolve conflict dengan asal pilih	Pahami maksud kedua perubahan dan test lagi.
Force push sembarangan	Hindari; ikuti policy dan pahami rebase.
Merge walau CI merah	Perbaiki checks lebih dulu.
Terus coding di branch yang sudah merged	Kembali ke main terbaru dan buat branch baru.

23. Command harian yang perlu benar-benar hafal

```
# lihat kondisi
git status
git diff
git diff --staged

# branch
git branch
git switch main
git switch -c feature/APP-123-something

# sinkronisasi
git fetch origin
git pull origin main

# simpan perubahan
git add <file>
git commit -m "feat: ..."
git push

# history
git log --oneline --graph --decorate --all

# integrasi
git merge origin/main
git rebase origin/main

# cleanup
git branch -d <branch>
```

24. Bedakan command yang sering tertukar

Istilah	Makna praktis
git add	Memilih perubahan untuk commit berikutnya.
git commit	Menyimpan snapshot/perubahan ke history lokal.
git push	Mengirim commit lokal ke remote.
git fetch	Mengambil informasi/commit remote tanpa langsung mengintegrasikan ke branch aktif.
git pull	Mengambil remote lalu mengintegrasikan sesuai konfigurasi.
Pull Request	Proposal kolaborasi untuk menggabungkan perubahan branch; bukan command git pull.
merge	Menggabungkan history dua garis pengembangan.
rebase	Memindahkan/replay commit ke base baru; menulis ulang commit yang direplay.
revert	Membuat commit baru yang membalik efek commit sebelumnya.
reset	Memindahkan pointer/history lokal; dampaknya bergantung mode dan harus digunakan hati-hati.

25. Latihan mandiri level perusahaan kecil

Setelah APP-101 dan APP-102 selesai, ulangi workflow tanpa melihat halaman command. Buat tiga ticket: APP-103 add location field, APP-104 add profile formatter, dan APP-105 fix empty bio handling. Untuk setiap ticket, buat branch sendiri, commit terarah, PR sendiri, dan review dari Akun A.

Kemudian sengaja buat APP-103 dan APP-104 mengubah bagian file yang sama agar kamu mengalami branch divergence. Tujuannya bukan membuat conflict sebanyak mungkin, tetapi belajar membaca graph dan memahami kenapa conflict muncul.

Kriteria lulus latihan

- [] Bisa menjelaskan local vs remote
- [] Bisa membuat branch dari main terbaru
- [] Selalu cek status/diff sebelum commit
- [] Bisa push dan membuka PR dengan base yang benar
- [] Bisa menangani Request Changes tanpa PR baru
- [] Bisa membaca checks
- [] Bisa menjelaskan merge vs rebase
- [] Bisa menyelesaikan conflict sederhana dan test ulang
- [] Bisa melakukan squash merge
- [] Bisa sync main dan cleanup branch setelah merge

26. Workflow ringkas untuk ditempel di meja

MULAI TASK

```
git switch main  
git pull origin main  
git switch -c feature/TICKET-deskripsi
```

KERJA

```
edit -> test -> git status -> git diff  
git add <file>  
git diff --staged  
git commit -m "feat/fix: ..."
```

KIRIM

```
git push -u origin <branch>  
buka PR -> self review -> request review
```

REVIEW

```
request changes?  
edit -> test -> commit -> push pada branch yang sama
```

SEBELUM MERGE

```
checks hijau  
review approved  
conversations selesai  
conflict selesai
```

SELESAI

```
merge sesuai policy tim  
git switch main  
git pull origin main  
hapus branch lama
```

Kalau kamu sudah bisa menjalankan alur ini tanpa menghafal secara mekanis - dan bisa menjelaskan kenapa setiap langkah dilakukan - kamu sudah memiliki fondasi Git/GitHub collaboration yang kuat untuk masuk ke latihan project tim yang lebih besar.